

Practical 4

Implementation and Time Analysis of Factorial Program Using Iterative and Recursive Methods

1. Introduction

The factorial of a non-negative integer n is the product of all positive integers from 1 to n . It is represented by $n!$.

Formula:

- $0! = 1$
- $n! = n \times (n-1) \times (n-2) \times \dots \times 1$

Factorials are widely used in mathematics, permutations, combinations, probability, and algorithm design.

2. Aim

To implement the factorial of a number using both iterative and recursive methods and analyze their time and space complexities.

3. Objective

- To understand iterative and recursive approaches.
- To compute the factorial of a given number.
- To compare the performance of both methods. □ To analyze their time and space complexities.

4. Iterative Method

Algorithm

1. Read the number n .
2. Initialize $fact = 1$.
3. Repeat from 1 to n .
4. Multiply $fact$ by the current number.

5. Print the factorial.

Pseudocode

START

Read n

fact = 1

FOR i = 1 TO n

 fact = fact × i

ENDFOR

Print fact

STOP

5. Recursive Method

Algorithm

1. Read the number n.
2. If n is 0 or 1, return 1.
3. Otherwise return $n \times \text{factorial}(n-1)$.
4. Print the factorial.

Pseudocode

START

FUNCTION factorial(n)

IF n == 0 OR n == 1

 RETURN 1

ELSE

 RETURN $n \times \text{factorial}(n-1)$

ENDIF

Read n

Print factorial(n)

STOP

Code:

Using Iterative Method

```
n = int(input("Enter a number: "))  
  
fact = 1  
  
for i in range(1, n + 1):  
    fact *= i  
  
print("Factorial =", fact)
```

Enter a number: 5
Factorial = 120

Using Recursive Method

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial(n - 1)  
  
n = int(input("Enter a number: "))  
  
print("Factorial =", factorial(n))
```

Enter a number: 6
Factorial = 720

6. Time and Space Complexity Comparison

Method	Best Case	Average Case	Worst Case	Space Complexity
Iterative Factorial	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Recursive Factorial	$O(n)$	$O(n)$	$O(n)$	$O(n)$

7. Advantages

Iterative Method

- Easy to implement.
- Uses constant memory.
- Faster than recursion due to no function call overhead.

Recursive Method

- Simple and elegant code.
- Easy to understand recursive concepts.
- Suitable for recursive mathematical problems.

8. Disadvantages

Iterative Method

- Code can become lengthy for complex recursive problems. □ Less intuitive for recursive algorithms.

Recursive Method

- Uses extra memory due to the function call stack.
- May cause stack overflow for very large values of n .

9. Conclusion

The factorial of a number can be calculated using both iterative and recursive methods. Both approaches require $O(n)$ time. However, the iterative method is more memory efficient with $O(1)$ space complexity, while the recursive method requires $O(n)$ space because of recursive function calls. Therefore, the iterative approach is generally preferred for computing factorials, especially for large input values.